

Cenário de Observabilidade e Monitoração End-to-End

Projeto Integrador VI — FoodClub

FATEC | Tecnologia em Desenvolvimento de Software Multiplataforma | 6º Semestre / 2026

Prof. Dr. Cassio R. F. Riedo

1. Contexto do Sistema PI

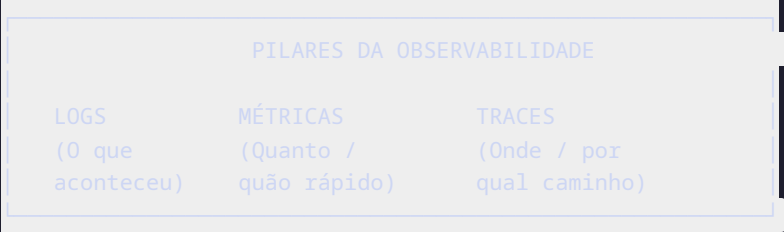
O **FoodClub** é uma plataforma B2B de alimentação corporativa que conecta restaurantes, empresas e funcionários. A arquitetura adotada é **cliente-servidor**, com separação clara entre:

Camada	Tecnologia	Hospedagem
Frontend Mobile	React Native + Expo SDK 54 + NativeWind	Dispositivo do usuário (Android/iOS)
API Backend	NestJS 10 + TypeScript (Clean Architecture)	Azure Container Apps
Banco de Dados	PostgreSQL 15 (Sequelize ORM)	Azure Database for PostgreSQL Flexible Server
Pipeline CI/CD	GitHub Actions	GitHub / Azure Container Registry
PLN	TF-IDF + SVM (chatbot) + expo-speech-recognition (voz)	Client-side

O processo de automatização do PI abrange: deploy automatizado via GitHub Actions, testes unitários com Jest (cobertura 95,04%), probe de saúde (GET /health-check) a cada 60 s e monitoramento via **AWS CloudWatch** (CloudWatchLoggerService + CloudWatchMetricsService) e **Azure Monitor** para recursos de infraestrutura.

2. O que é Observabilidade End-to-End

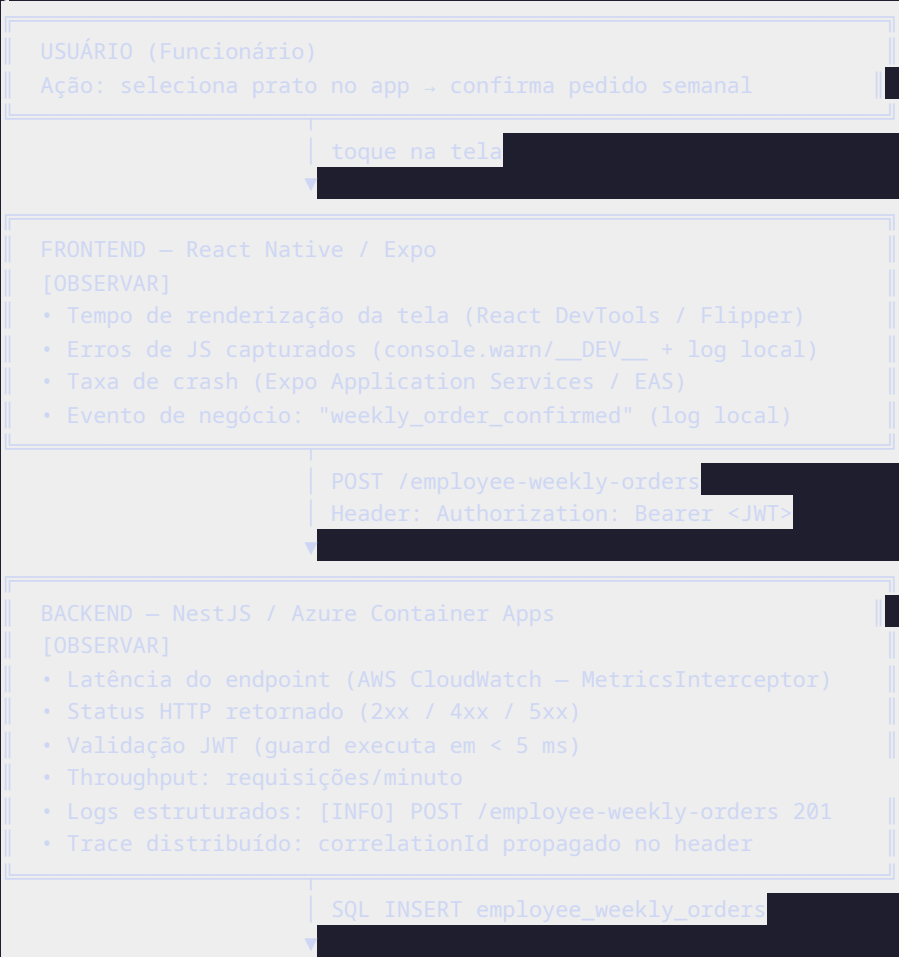
Observabilidade E2E é a capacidade de entender o comportamento interno de um sistema a partir de suas saídas externas. Ela é sustentada por três pilares:

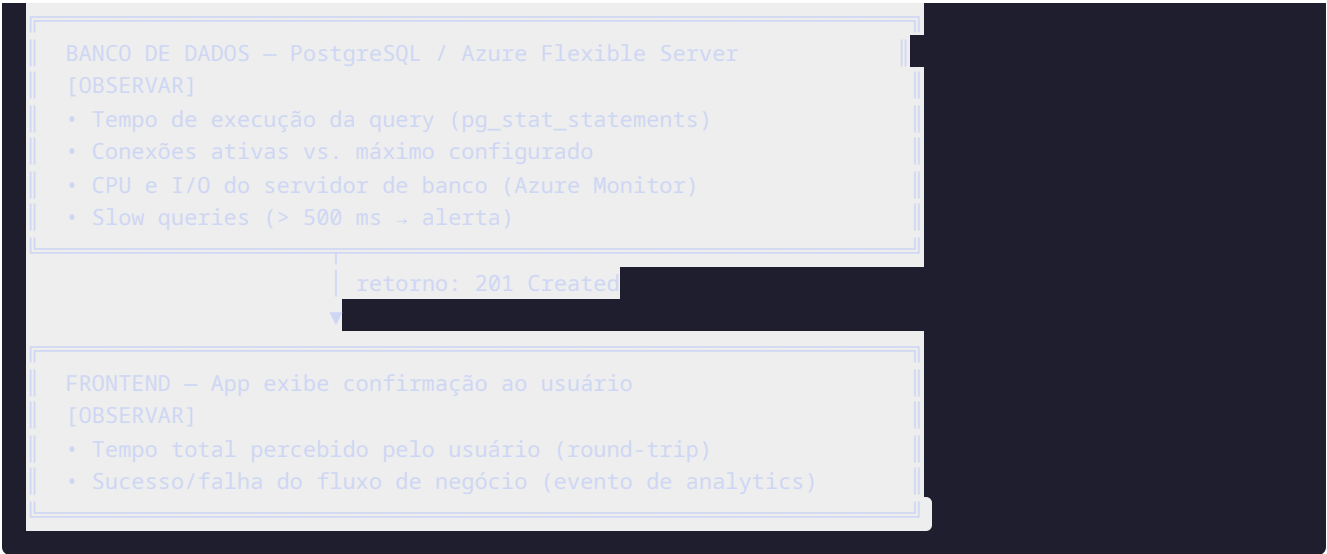


No FoodClub, cada pilar cobre todas as camadas — do toque do usuário no app até o retorno do banco de dados.

3. Fluxo End-to-End de Observabilidade

O diagrama abaixo representa o caminho completo de uma requisição crítica (ex.: funcionário realiza pedido semanal) e os pontos de instrumentação em cada etapa:





4. Observabilidade no Frontend (React Native / Expo)

4.1. O que monitorar

Sinal	Ferramenta	Critério de Alerta
Crashes e erros não tratados	Expo Application Services (EAS) / console.warn + __DEV__	Taxa de crash > 0,5% das sessões
Performance de tela (Time to Interactive)	Flipper / React DevTools Profiler	Renderização > 300 ms
Falha na busca por voz	Log local (__DEV__ console.warn)	expo-speech-recognition retorna erro
Latência de chamadas HTTP (Axios)	Interceptor Axios (recomendação: ampliar para rastrear latência)	p95 > 2 s
Eventos de negócio	Log local (sem SDK de analytics implementado atualmente)	Funil: login → busca → pedido

4.2. Instrumentação do Interceptor Axios (já existente — ampliar)

O baseRepository.ts já possui interceptor para logout automático em 401. Acrescentar rastreamento de latência:

```
// Exemplo de instrumentação de latência no interceptor existente
axiosInstance.interceptors.request.use((config) => {
  config.metadata = { startTime: Date.now() };
  return config;
});

axiosInstance.interceptors.response.use((response) => {
  const duration = Date.now() - response.config.metadata.startTime;
  // Enviar para telemetria: endpoint + duration + statusCode
  trackMetric('api_latency', duration, {
    endpoint: response.config.url,
    status: response.status,
  });
  return response;
});
```

4.3. Mapa de Jornada Monitorada (Funcionário)

```
Login → [obs: tempo JWT + 401/403]
  → Home (lista restaurantes) → [obs: tempo carregamento FlatList]
  → Voz/Busca → [obs: taxa de reconhecimento correto]
    → Detalhe restaurante → [obs: tempo GET /Restaurant/:id]
      → Seleção semanal → [obs: POST /employee-weekly-orders]
        → Confirmação → [obs: evento "order_placed" registrado]
```

5. Observabilidade no Backend (NestJS / Azure Container Apps)

5.1. Health Check — Endpoint Atual

O FoodClub já possui `GET /health-check` (controller em `src/interfaces/http/controllers/health-check.controller.ts`), que retorna:

```
{
  "status": "Is Alive!",
  "timestamp": "2026-05-27T10:00:00.000Z"
}
```

```
"uptime": 3600.5
```

Expansão recomendada — Health Check profundo (deep health):

```
// Expandir para verificar conectividade com o banco
@Get('deep')
async deepHealthCheck() {
  const dbStatus = await this.checkDatabaseConnection();
  return {
    status: dbStatus ? 'healthy' : 'degraded',
    timestamp: new Date().toISOString(),
    uptime: process.uptime(),
    dependencies: {
      database: dbStatus ? 'up' : 'down',
    }
  }
}
```

5.2. Métricas de API — Red Method (Rate, Errors, Duration)

Métrica	Descrição	Meta (RNF01)
Rate (Requisições/min)	Volume de tráfego por endpoint	Baseline: 200 req/min
Errors (Taxa de erro)	% de respostas 5xx	< 1%
Duration (Latência)	p50, p95, p99 por endpoint	p95 < 2 s

Endpoints críticos para instrumentar (mapeados nos RFs):

- POST /user/login (RF03) — autenticação
- GET /Restaurant (RF07) — lista restaurantes
- POST /employee-weekly-orders (RF10) — pedido semanal
- GET /health-check — disponibilidade (RNF04)

5.3. Logs Estruturados

O backend já possui `CloudWatchLoggerService` que envia logs para AWS CloudWatch. O formato JSON estruturado atual é:

```
// Formato de log estruturado (JSON)
```

```
{
  "level": "INFO",
  "timestamp": "2026-05-27T10:15:00Z",
  "correlationId": "abc-123",
  "method": "POST",
  "path": "/employee-weekly-orders",
  "statusCode": 201,
  "durationMs": 87,
  "userId": 42,
  "employeeId": 15
}
```

5.4. Rastreamento Distribuído (Traces)

O backend já registra `correlationId` via `AuditInterceptor` (`src/infrastructure/observability/audit.interceptor.ts`). A recomendação é propagar esse ID também no header das requisições mobile:

```
App Mobile -> [X-Correlation-ID: uuid] -> NestJS (AuditInterceptor) -> [log correlationId no CloudWatch] -> PostgreSQL
```

Isso permite reconstruir a jornada completa de uma requisição com falha nos logs do AWS CloudWatch.

6. Observabilidade da Infraestrutura (Azure)

6.1. Recursos Monitorados

Recurso	Ferramenta	Métricas Principais
Azure Container Apps (API NestJS)	Azure Monitor	CPU %, Memória %, Réplicas ativas, Tempo de cold start
Azure Database for PostgreSQL	Azure Monitor	Conexões ativas, CPU %, Storage usado, Latência de query
Azure Container Registry	Azure Monitor	Pull/push rate, Espaço utilizado
Azure Key Vault	Audit Logs	Acessos a secrets (quem, quando)

6.2. Configuração de Alertas (já parcialmente configurado)

Conforme documentação técnica (seção 7.3), os alertas existentes são:

Condição	Ação	Threshold
p95 de latência > 3 s	Email automático	AWS CloudWatch Alarm
Taxa de erro > 5%	Email automático	AWS CloudWatch Alarm
Falha no pipeline CI/CD	Email via GitHub Actions notify job	—

Alertas adicionais recomendados:

Condição	Ação	Prioridade
GET /health-check retorna falha por 3 probes consecutivos	PagerDuty / Teams	Crítica
Conexões PostgreSQL > 80% do pool máximo	Email	Alta
CPU Container App > 85% por 5 min	Escalar réplica automaticamente	Alta
Cobertura de testes cai abaixo de 80% no pipeline	Bloquear merge PR	Média

7. Observabilidade do Pipeline CI/CD (DevOps)

7.1. Fluxo de Automação Atual

```
Git push → GitHub Actions
├─ validate: npm ci + lint (Husky) + type check
├─ test:     Jest (cobertura 95,04%)
├─ docker:   build imagem → push ACR (acrfoodclub)
└─ deploy:   Azure Container Apps (HML ou PRD)
```

7.2. O que monitorar no Pipeline

Etapa	Métrica	Alerta
validate	Tempo de execução	> 5 min → investigar
test	Cobertura de código	< 80% → bloquear deploy
test	Testes falhos	Qualquer falha → bloquear + notificar
docker	Tamanho da imagem	> 500 MB → otimizar
deploy	Tempo de deploy	> 10 min → investigar
deploy	Taxa de sucesso	< 95% nas últimas 10 runs → alerta

7.3. DevTest — Rastreabilidade de Qualidade

Cada build do pipeline gera evidências rastreáveis:

```
Commit SHA → Relatório Jest (coverage/lcov-report/index.html)
              → Log de validação de lint
              → Log de build Docker
              → Status do deploy (HML/PRD)
              → URL do ambiente publicado
```

8. DevSecOps — Observabilidade de Segurança

Ponto de Controle	O que verificar	Ferramenta
JWT inválido	Requisições com token expirado ou malformado (401)	AWS CloudWatch — filtro por status 401 nos logs
Brute force login	Múltiplas tentativas POST /user/login com 401 no mesmo IP	Log Analytics query + alerta
Exposição de dados sensíveis	Respostas da API contendo CPF/CNPJ/senha	Teste automatizado no pipeline (grep no response body)
Secrets no código	Chaves e passwords commitadas	GitHub Secret Scanning (nativo)
Vulnerabilidades de dependências	Pacotes npm com CVE conhecidas	npm audit no step validate do pipeline
Acesso ao Key Vault	Quem acessou qual secret e quando	Azure Key Vault Audit Logs → Log Analytics

9. AIOps — Monitoração Inteligente

AIOps aplica Inteligência Artificial sobre os dados de observabilidade para detectar anomalias e prever falhas antes que o usuário seja impactado.

9.1. Aplicação no FoodClub

Cenário	Dado de Entrada	Ação Inteligente
Anomaly Detection	Latência média do GET /Restaurant	AWS CloudWatch Anomaly Detection — alerta automático se latência desviar $> 2\sigma$ da baseline
Failure Prediction	Aumento progressivo de erros 5xx no backend	AWS CloudWatch Metrics + alertas preditivos
Capacity Planning	Pico de requisições toda segunda-feira (pedidos semanais)	Análise de série temporal → escalar Container Apps proativamente
Root Cause Analysis	Cadeia de erros correlacionados (DB lento → timeout backend → erro app)	AWS CloudWatch Logs Insights — correlação por correlationId

9.2. Relação com o Módulo PLN do FoodClub

O chatbot FAQ (TF-IDF + SVM) e a busca por voz (expo-speech-recognition) são módulos de IA do próprio produto. Para o AIOps, monitora-se:

Módulo	Métrica	Meta
Chatbot FAQ	% de mensagens sem correspondência (fallback)	$< 20\%$ → se acima, expandir constants/faq.ts
Busca por voz	Taxa de transcrição bem-sucedida	$> 90\%$
Classificador SVM	Acurácia por categoria de intenção	F1-score $> 0,85$ por categoria

10. MLOps — Ciclo de Vida dos Modelos de PLN

O FoodClub possui dois componentes de Machine Learning:

- 1. **Classificador SVM** (chatbot — TF-IDF + SVM, backend ou client-side)
- 2. **Reconhecimento de voz** (API nativa Android/iOS via expo-speech-recognition)

O MLOps define como esses modelos são versionados, monitorados e atualizados:

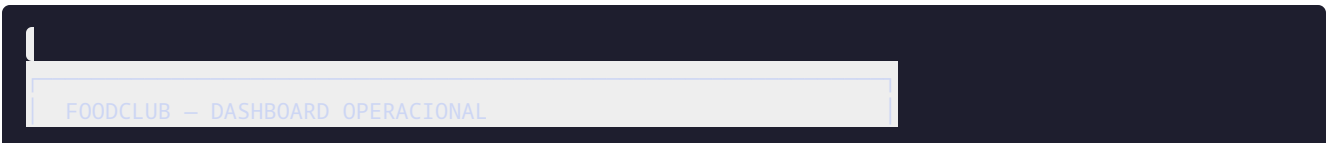


Monitoramento do modelo em produção:

Sinal	Como capturar	Alerta
% de respostas fallback do chatbot	Contador no client-side enviado ao backend	> 20% → revisão do dataset
Categorias mais consultadas	Log de intenções classificadas	Base para expandir FAQ
Degradação do F1-score	Re-avaliação manual a cada sprint	F1 < 0,80 → re-treino

11. Dashboard de Monitoração — Visão Consolidada

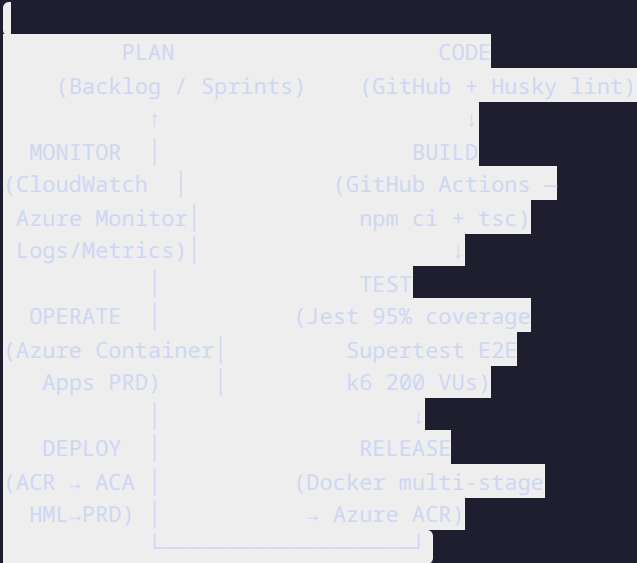
Painel Operacional (AWS CloudWatch / Azure Monitor)



API HEALTH ✔ Healthy	LATÊNCIA p95 187 ms	TAXA ERRO 0,3%	USUÁRIOS ATIVOS 42 sessões
ENDPOINTS MAIS LENTOS (últimas 24h)			
1. POST /company/:id/create-orders-from-weekly → 1.240 ms			
2. GET /Restaurant (sem cache) → 420 ms			
3. POST /user/login → 190 ms			
PIPELINE CI/CD (última execução)			
validate ✔ 1m12s test ✔ 2m48s deploy ✔ 4m03s			
Cobertura: 95,04% Branch: development → HML			
BANCO DE DADOS (Azure PostgreSQL)			
Conexões: 12/100 CPU: 8% Storage: 1,2 GB / 32 GB			
PLN – CHATBOT			
Mensagens hoje: 37 Fallback: 8,1% Top intent: "pedido"			

12. Modelo de Processo — DevOps Loop Completo

O FoodClub implementa o **DevOps Infinity Loop** com instrumentação de observabilidade em cada fase:



Integração DevSecOps

A segurança é integrada em cada etapa:

- **Plan:** ameaças mapeadas na documentação (OWASP relevante para JWT, SQL Injection)
- **Code:** bcrypt para senhas, class-validator para DTOs, CNPJ não exposto
- **Build:** npm audit no pipeline
- **Test:** testes de autenticação (401/403), validação de dados sensíveis
- **Release:** secrets no Azure Key Vault, Managed Identity
- **Operate:** logs de acesso ao Key Vault, alertas de 401 em massa
- **Monitor:** anomaly detection em padrões de autenticação

13. Critérios de Aceite da Observabilidade

Critério	Meta	Status
Health check disponível e respondendo	GET /health-check retorna 200 em < 200 ms	✓ Implementado
Logs estruturados no backend	JSON com timestamp, path, status, duration	↔ A ampliar
Alertas de latência configurados	Email se p95 > 3 s	✓ Configurado (CloudWatch Alarm)
Alertas de erro configurados	Email se erro > 5%	✓ Configurado (CloudWatch Alarm)
Monitoramento de pipeline	Notificação em caso de falha	✓ GitHub Actions notify job
Dashboard centralizado	AWS CloudWatch / Azure Monitor	✓ Parcialmente configurado
Rastreamento de segurança	Logs de 401 + Key Vault audit	↔ A configurar
Monitoramento do chatbot	% fallback registrado	↔ A implementar

14. Referências

- AMAZON WEB SERVICES. **Amazon CloudWatch documentation**. Disponível em: docs.aws.amazon.com/cloudwatch.
- MICROSOFT. **Azure Monitor documentation**. Disponível em: learn.microsoft.com/azure/azure-monitor.
- NESTJS. **Logger**. Disponível em: docs.nestjs.com/techniques/logger.
- KIM, G. et al. **The DevOps Handbook**. IT Revolution Press, 2016.
- SRIDHARAN, C. **Distributed Systems Observability**. O'Reilly Media, 2018.
- GOOGLE SRE. **Site Reliability Engineering**. Disponível em: sre.google/sre-book.

Equipe: Alinne Martins · Bruno Pasqual · Matheus Prado · Matheus Fernandes · Fabricio Soica

Elaboração: 27/05/2026